Appello del 29 Giugno 2018



Politecnico di Milano

Anno accademico 2017-2018

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica della classe `CodaConRitardo`, che implementa l'interfaccia `Collection` (ed ha quindi anche i metodi `contains, containsAll, isEmpty, removeAll, toString, size`, ...). La classe definisce una coda la cui testa è l'elemento più in ritardo rispetto alla propria scadenza, ossia quello scaduto da più tempo. Se nessun elemento è ancora scaduto, la *testa non esiste* e il metodo `poll()` restituisce `null`. Si suppone che la coda non ammetta elementi duplicati, ossia lo stesso oggetto non può essere inserito più volte nella coda.

```
public class CodaConScadenze<E> implements Collection<E> {

  /* costruisce una coda vuota */
  public CodaConScadenze()

  /* inserisce x nella coda se non e' gia' presente (la scadenza è indicata
     in ms dal tempo attuale);
   * restituisce false sse x e' gia' nella coda  */
  public boolean add(E x, long scade)

  /* rimuove tutti gli elementi: */
  public void clear()

  /* ritorna null se la coda e' vuota, altrimenti  restituisce, senza
     eliminarla, a testa della coda;
   * se la testa non esiste, restituisce l'elemento con scadenza più prossima  */
  public E peek()

  /* restituisce ed elimina la testa della coda;
   * ritorna null (senza eliminare nulla) se la testa non esiste
   * (secondo la definizione). */
  public E poll()

  /* restituisce un iteratore su tutti gli elementi (scaduti o no) della coda,
   * in ordine di scadenza.
   * La coda non puo' essere modificata mentre un iteratore e' in uso  */
  public  Iterator<E>iterator()

  /* restituisce un arraylist che contiene tutti gli elementi della coda, *
   * nell'ordine di scadenza  */
  ArrayList<E> toArrayList()

  /* restituisce il valore della scadenza dell'elemento x al momento della
   * chiamata. Se l'elemento non e' presente nella coda, lancia un'eccezione */
  public long scadenza(E x) throws NotFoundException

}
```

Domanda a

Si stabilisca preliminarmente quali metodi sono osservatori. Specificare quindi in JML un invariante pubblico per la classe. Vi sono proprieta' interessanti della classe che non si possono esprimere con un invariante?

Si specifichino i metodi `add(), peek(), poll()` in funzione (di alcuni) degli osservatori.

Soluzione

Gli osservatori sono `toArrayList()`, `scadenza()` e `peek()`, oltre naturalmente ai metodi di `Collection`: `contains`, `containsAll`, ecc.

L'invariante pubblico stabilisce che non vi sono duplicati:

```
public invariant
  (\forall int i; 0<= i && i<this.size()-1;
       (\forall int j; i< j && j<this.size() x;
     toArrayList.get(i)!=toArrayList.get(j);
```

La proprieta' che l'ArrayList restituito toArrayList e' in ordine di scadenza e' garantita nella postcondizione del metodo, ma non e' errato richiamarla nell'invariante.

Le altre proprieta' tipiche di questa coda sono di tipo evolutivo (ad esempio, tutte le scadenze procedono con lo stesso ritmo) e non possono quindi essere espresse con un invariante.

```
//@ensures
//@\result==(toArrayList().size()==0 ? null
//@         : toArrayList().get(0));
public /*@ pure */ E peek()

//@ensures
//@\result==!this.contains(e) &&
//@ this.contains(e) && size()==\old(size()+1) &&
//@ this.containsAll(\old(toArrayList())
//@ scadenza(x) == scade;
public boolean add(E x, long scade)

//@ensures
//@ toArrayList().size()==0 || scadenza(toArrayList().get(0))>=0
//@ ? (\result== null && toArrayList().equals(\old(ArrayList()))
//@ : (\result== toArrayList().get(0) &&
//@    !this.contains(\result) && size()==\old(size()-1) &&
//@    \old(toArrayList()).equals(toArrayList().subList(1,toArrayList.size()-1)));
  public E poll()
```

Domanda b

Si consideri la seguente implementazione (parziale) della classe `CodaConScadenze`, che non è ottimizzata dal punto di vista dell'efficienza.

Gli elementi della coda sono memorizzati in un `ArrayList<E> elems` in un ordine qualunque.

Un `ArrayList<Long> timeStamps` contiene in posizione `i` la scadenza attuale per l'elemento `elemens.get(i)`, ovvero il tempo in ms che deve passare prima che l'elemento sia considerato scaduto.

I valori dei tempi sono aggiornati ogni volta che si svolge un'operazione che modifica la coda, come `add` e `poll`, attraverso la chiamata di l'opportuno metodo privato `updateTimeStamps()`. Per implementare questo metodo si è supposta, per semplicità, l'esistenza di una classe di sistema `Time`, il cui metodo `elapsed()` restituisce il tempo in ms passato dall'ultimo azzeramento, ottenuto chiamando il metodo `reset()`.

Un frammento del codice e' mostrato di seguito.

```
public class CodaConScadenze<E> implements Collection<E> {
  private  ArrayList<E> elems;
  private Time lastUpdate;
  private ArrayList<Long> timeStamps;

  public CodaConScadenze() {
     elems=new ArrayList<E>;
```

```
    timestamps = new ArrayList<Long>;
    Time lastUpdate = new Time();
    lastUpdate.reset();
  }
  private void updateTimeStamps() {
    long z = lastUpdate.elapsed();
    timestamps.forEach(t -> t-z);
    lastUpdate.reset();
  }
  public boolean add(E x, long scadenza) {
    if elems.contains(x) return false;
    updateTimeStamps();
    elems.add(x);
    timestamps.add(scadenza);
    return true;
  }
  ...
}
```

Si scriva l'invariante di rappresentazione della classe e la funzione di astrazione. Si implementi, eventualmente aiutandosi con commenti, il metodo `poll()`.

Soluzione

RI:

```
private invariant elems!= null && lastUpdate &&  timeStamps!= null &&
!elems.contains(null) && !timeStamps.contains(null) &&
elems.size()== timeStamps.size();
```

AF:

```
private invariant elems!= null && lastUpdate &&  timeStamps!= null &&
(\forall int i; 0<= i && i<elems.size(); timeStamps.get(i)==elems.get(i).scadenza())
toArrayList().containsAll(elems) && toArrayList().size()==elems.size() &&
(\forall int j; 0<= i && j<elems.size()-1;
   toArrayList().get(j).scadenza()<=toArrayList().get(j+1).scadenza());
```

Implementazione di poll: Sia `min(ArrayList<Long> x)` un metodo che restituisce l'indice del minimo elemento contenuto in x.

```
public E poll() {
  if (elems.size()==0) return null;
  long z = min(timeStamps);
  if (z>0) return null;
  x = elems.get(z); //salva in x l'elemento da eliminare;
  /*rimuove elemento in posizione z, sia da elems che da timeStamps: */
  elems.set(z,elems.get(elems.size()-1);
  timeStamps.set(z,timestamps.get(elems.size()-1);
  elems.remove(elems.size()-1);
  timeStamps.remove(elems.size()-1);
  updateTimeStamps();
}
```

Esercizio 2

Si consideri la seguente classe Java che realizza il tipo `Buffer` capace di contenere un massimo di 10 elementi interi positivi. Il valore convenzionale -1 è usato internamente per identificare un elemento vuoto. Il metodo `get(pos)` restituisce il contenuto dell'elemento in posizione `pos`, se presente, altrimenti solleva l'eccezione `EmptyElementException`. Il metodo `set(pos, val)` inserisce il valore `val` in posizione `pos`, sovrascrivendo un eventuale valore già presente.

```
public class Buffer {
    private int[] data;

    public Buffer() {
        data = new int[10];
        for(int i=0; i<10; i++) data[i]=-1;
    }

    public int get(int pos) throws EmptyElementException {
        if(data[pos]==-1) throw new EmptyElementException();
        int x = data[pos];
        data[pos] = -1;
        return x;
    }

    public void set(int pos, int val) {
        data[pos] = val;
    }
}
```

Domanda a

Si modifichi la classe `Buffer` in maniera che il metodo `get(pos)` sospenda il chiamante se l'elemento in posizione `pos` è vuoto, risvegliandolo quando l'elemento richiesto viene inserito. Analogamente, il metodo `set(pos, val)` deve sospendere il chiamante se l'elemento in posizione `pos` è già pieno, in attesa che si svuoti. Si usino solo i meccanismi base della sincronizzazione di Java senza sfruttare le classi dei pacchetti `java.util.concurrent.XXX`.

Soluzione

```
public class SyncBuffer {
    private int[] data;

    public SyncBuffer() {
        data = new int[10];
        for(int i=0; i<10; i++) data[i]=-1;
    }

    public synchronized int get(int pos) throws InterruptedException {
        while(data[pos]==-1) wait();
        int x = data[pos];
        data[pos] = -1;
        notifyAll();
        return x;
    }

    public synchronized void set(int pos, int val) throws InterruptedException  {
        while(data[pos] != -1) wait();
        data[pos] = val;
        notifyAll();
    }
}
```

Domanda b

Si aggiunga alla classe sviluppata al punto precedente un metodo `public void setAll(int val)` che imposta tutti gli alementi del buffer al valore `val` (anche se già pieni). Il metodo deve eseguire in maniera asincrona rispetto al chiamante, ovvero il chiamante deve ritornare immediatamente mentre il metodo esegue in un thread separato.

Soluzione

Si noti l'uso della classe anonima per creare il thread che esegue in maniera asincrona il corpo del metodo `setAll`, nonchè l'uso della notazione `SyncBuffer.this` per riferirsi all'istanza della classe `SyncBuffer` onde sincronizzarsi correttamente.

```
public class SyncBuffer {
    ...
    public void setAll(int val) {
        new Thread() {
            public void run() {
                synchronized(SyncBuffer.this) {
                    for(int i=0; i<10; i++) data[i] = val;
                    SyncBuffer.this.notifyAll();
                }
            }
        }.start();
    }
}
```

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class Operatore {
    public abstract void calcola(double a, double b);
}

class Prodotto extends Operatore {
    public void calcola(double a, double b) { System.out.println("Prodotto: "+ (a*b)); }
}

class Somma extends Operatore {
    public void calcola(double a, double b) { System.out.println("Somma: "+ (a+b)); }
    public void calcola(int a, int b) { System.out.println("Somma int: "+ (a+b)); }
}

class SommaDoppia extends Somma {
        public void calcola(int a, int b) { System.out.println("Somma doppia int: "+ (a+b)*2);
}
```

Si consideri poi il seguente frammento di codice Java che utilizza le classi sopra.

```
1.        Operatore op1, op2;
2.        SommaDoppia op3, op4;

3.        op1 = new Somma();
4.        op2 = new Prodotto();
5.        op3 = new SommaDoppia();
6.        op4 = new Somma();

7.        op1.calcola(3.0,5.0);
```

```
8.        op2.calcola(3.0,5.0);
9.        op1.calcola(3,5);
10.       op3.calcola(3,5);
11.       op4.calcola(3,5);

12.       op4 = (SommaDoppia) op3;
13.       op4.calcola(3.0,5.0);
14.       op4 = (SommaDoppia) op1;
15.       op4.calcola(3,5);
16.       op1 = op2;
17.       op1.calcola(3.0,5.0);
18.       op1.calcola(3,5);
```

Domanda a

Si indichino dapprima quali righe del metodo `main` generano un errore di compilazione e quali un errore a tempo di esecuzione, motivando la propria risposta.

Soluzione

La riga 6 non compila essendo il tipo dinamico della variabile op4 (`Somma`) una sopraclasse (e non una sottoclasse) del tipo statico (`SommaDoppia`)

La riga 11 va eliminata come conseguenza del punto precedente (errore: "variable op4 might not have been initialized").

Infine, la riga 14 compila correttamente (il compilatore "si fida" del nostro cast) ma viene restituita un'eccezione a tempo di esecuzione ("java.lang.ClassCastException: Somma cannot be cast to SommaDoppia") perchè l'oggetto il cui riferimento è contenuto in `op1` non è di tipo `SommaDoppia` nè di una sottoclasse (è di tipo `Somma`).

Domanda b

Supponendo che tutte le righe che provocano errori in fase di compilazione e/o di esecuzione siano state rimosse, illustrare l'output del programma, motivando brevemente la propria risposta.

Soluzione

```
7.   Somma: 8.0
8.   Prodotto: 15.0
9.   Somma: 8.0
10.  Somma doppia int: 16
13.  Somma: 8.0
15.  Somma doppia int: 16
17.  Prodotto: 15.0
18.  Prodotto: 15.0
```

Le prime due stampe sono ovvie.

La riga 9 invoca il metodo `calcola(double, double)` e non il metodo `calcola(int, int)` essendo la scelta del metodo da invocare (signature) effettuata a tempo di compilazione, guardando solo al tipo statico e il tipo statico di `op1` è `Operatore`, che ha il solo metodo `calcola(double, double)`. Il codice effettivamente eseguito è poi determinato a tempo di esecuzione e in questo caso viene eseguito il codice del metodo `calcola(double, double)` di `Somma`,

La stampa relativa alla riga 10 è ovvia (tipo statico e dinamico di `op3` sono `SommaDoppia`).

Alla riga 13 il tipo statico e quello dinamico della variabile `op4` sono entrambi `SommaDoppia`, classe che a sua volta possiede entrambi i metodi `calcola`. In questo caso il compilatore (sulla base del tipo dei parametri) sceglie la versione `double` ereditata dalla classe `Somma` e non ridefinita.

La riga 15, visto che la riga 14 non deve essere considerata (si veda la risposta precedente), vede ancora una volta tipo statico e dinamico `SommaDoppia`. In questo caso il compilatore sceglie la versione `int`, ridefinita in `SommaDoppia`.

Le stampa delle righe 17 e 18 sono ovvie: tipo statico `Operatore`, tipo dinamico `Prodotto`, unico metodo `calcola(double, double)`, quindi conversione automatica del tipo dei parametri da `int` a `double` alla riga 18.

Esercizio 4

Si consideri il seguente frammento di codice, espresso in pseudocodice:

```
READ foo
READ boo
IF boo > 12 OR foo < 70 THEN
   PRINT "pippo "
END IF
PRINT "pluto "
IF foo > 75 THEN
   PRINT "topolino!"
END IF
```

e si indichi quale tra copertura delle istruzioni, copertura dei cammini, copertura delle diramazioni (branch), e copertura delle condizioni viene massimizzata da ciascuna delle seguenti test suite. La risposta potrebbe non essere univoca.

1. boo = 13, foo = 77
2. boo = 14, foo = 74
3. boo = 10, foo = 80; boo = 13, foo = 80
4. boo = 10, foo = 80; boo = 13, foo = 80; boo = 13, foo = 73
5. boo = 10, foo = 80; boo = 13, foo = 80; boo = 9, foo = 60

Si indichi inoltre se si ritiene necessario modificare una test suite o aggiungerne una ulteriore per migliorare la copertura totale ottenuta dalle cinque suite sopra. Se sì, indicate quale suite si ritiene di voler modificare o aggiungere, e perchè.

Soluzione

Per ciascuna delle test suite proposte, vale il seguente:

1. Ottiene copertura 100% per le istruzioni.
2. Nessuna copertura viene massimizzata.
3. Ottiene copertura 100% per le istruzioni; notare che la copertura delle diramazioni rimane incompleta.
4. Ottiene copertura 100% per le istruzioni e per le diramazioni.
5. Ottiene copertura 100% per le istruzioni e per le condizioni; quest'ultima include le diramazioni.

Considerate queste test suite, l'unico criterio che non è mai massimizzato è quello dei cammini. Per avere 100% di copertura in quel caso, è sufficiente ad esempio aggiungere un test case che copra entrambi i rami `false` alla test suite 4., come boo=10, foo = 72.